FORCE-AWARE PEG INSERTION PROJECT

Interview Defense

Hard questions, honest answers, and the evidence behind them

A 2×2 ablation of force observation vs. force penalty for PPO peg insertion
NVIDIA Isaac Lab 3.0 + Isaac Sim 6.0 · RL-Games PPO · Franka arm (`franka_mimic.usd`)
4 policies × 3 seeds × 500 epochs · 12 evaluated checkpoints · DGX Spark (GB10)

This document is preparation material, not a results paper. Every number in it is traceable to `docs/experiment_log.md`, `results/tables/`, or the code in `force_peg_rl/`. Where the project has not measured something, the answer says so — an interviewer who catches an overclaim will discount everything else, and the strongest asset this project has is that its own records contain two retracted wrong explanations and eight instrumentation bugs caught before they reached a conclusion.

Orientation: what this project is, right now

Before any question-and-answer, the state of the work. Getting this right in the first sixty seconds is what buys credibility for everything after it.

The 30-second version

"I forked NVIDIA's FORGE peg-insertion task in Isaac Lab and ran a controlled 2×2 ablation to separate two things that 'force-aware manipulation' usually bundles together: giving the policy a contact-force **observation**, and putting a contact-force **penalty** in the reward. Four policies, three seeds each, 500 PPO epochs per seed, then all twelve checkpoints evaluated on the same 500 deterministic episodes. The observation is what lets the policy learn the task at all — it roughly doubles success over the geometry-only baseline. The penalty on its own, without the observation, makes things worse than the baseline: a policy punished for contact it cannot sense learns to avoid inserting rather than to insert carefully."

The 3-minute version, with the caveats attached

Add, unprompted, the three things a sharp reviewer will otherwise find themselves: absolute success rates are low (7–14%, against the project's own 80% target); the evaluation suite is nominal-only, so nothing here yet measures the **robustness** half of the research question; and episodes never terminate early, which makes "force-limit rate" an incidence rate rather than a termination rate.

Status board

Area	State	Evidence / caveat
Fork of upstream FORGE	done	20-iter/64-env smoke test reproduced upstream's final reward **bit-identically** (22.362465). Intentionally broken later by adding `termination_reason` logging.
Ablation plumbing	done	`use_force_obs` / `use_force_penalty` toggles; verified by observation size (24 vs 20 actor dims) and a flat-zero `logs_rew_contact_penalty` curve, not by assumption.
Training: A, B, C, D	done	3 seeds each, 500 epochs, 512 envs, 33M env steps/seed, 13 h/seed on a shared DGX Spark.
Evaluation script (§17)	done	`evaluate_policy.py`: deterministic inference, per-episode CSV with all 20 §11.4 columns.
Nominal evaluation	done	12 checkpoints × 500 episodes, fixed seeds `[1000, 1001, 1002]`. Real §22 table filled.
Representative videos	done	One clean success + one genuine near-miss per policy, curated from each checkpoint's own eval batch.
OOD / robustness suites	not done	`pose_shift`, `low/high_friction`, `mass_shift`, `combined_ood` are commented-out stubs; the script raises `NotImplementedError`. Week 5 scope.
Jam detection	not done	No signal, no threshold. §22 "Jam Rate" is `TBD`, deliberately not backfilled with 0%.

Area	State	Evidence / caveat
Early termination	opt-in, off	Implemented as an experimental flag with measurement counters; not used for any reported result.
Curriculum learning (§10)	not done	Training uses FORGE's full randomization from step one.
Technical report, 2-min video	not done	Week 6 scope.
Sim-to-real	out of scope	Explicit non-goal for version 1.

The measured result, in one table

Policy	Force obs.	Force pen.	Nominal success	Peak force p95	Force-limit
A: geometry baseline	No	No	7.3% (6.4–7.8)	37.2 N	1.8%
B: observation only	Yes	No	**14.2%** (9.6–17.2)	34.7 N	1.0%
C: penalty only	No	Yes	3.4% (0.0–10.2)	**14.2 N**	0.1%
D: force-aware (FORGE default)	Yes	Yes	9.9% (9.2–10.8)	18.8 N	0.1%

Mean over 3 seeds; parenthesised range is the per-seed min–max, not a confidence interval. 500 deterministic episodes per seed (1500 per policy). "Force-limit" is the fraction of episodes in which contact force ever exceeded the 50 N placeholder threshold — no episode was ever **ended** by it.

1. Motivation and framing

Q1. Why peg insertion? Isn't this the most solved problem in manipulation?
Peg insertion is solved **in a fixture**, with known geometry, a calibrated part, and a hand-tuned search primitive — that is a 1980s result and I would not claim otherwise. What is not solved is doing it under uncertainty in the pose estimate, the friction, the mass, and the compliance of the controller itself, without re-tuning per part. That is the regime this project targets: the peg's starting pose is randomized every episode, the controller's proportional gains are perturbed by up to ±41% per episode, the action dead-zone is re-randomized every two seconds of sim time, and the policy's own view of the socket frame is corrupted with noise. The reason I chose it anyway is that it is the smallest task that contains every hard ingredient of contact-rich manipulation — alignment under uncertainty, intentional contact, jamming, force limits — while remaining cheap enough to run a full factorial ablation with seeds. It is a **measurement** platform, not a claim to have solved insertion.

Q2. Why does force information matter here at all? The critic already knows the exact hole pose.
That is exactly the question the project exists to answer, and I would say so plainly, because the naive intuition cuts the other way: if you already have exact relative geometry, force is arguably redundant. Three reasons it may not be. First, the **actor** does not have exact geometry — it sees fingertip position

relative to a deliberately noised estimate of the socket frame, while only the critic gets the privileged clean poses. Second, geometry tells you where things are, not what they are doing to each other: rim contact, a jammed two-point wedge, and free-space hovering can all look nearly identical in a 0.1 mm-resolution pose observation but are trivially distinct in the force signal. Third, the controller is itself randomized, so the mapping from commanded setpoint to realized motion changes per episode; force is the only channel through which the policy can sense that it is pushing and not moving. The measured answer supports this: adding the force observation roughly doubled success (7.3% → 14.2%).

Q3. What is novel here versus simply re-running FORGE?
The environment is not novel and I do not present it as such — FORGE is public, NVIDIA's, and it ships force-aware by default. The contribution is the controlled decomposition. FORGE's published configuration is my Policy D: force observations **and** force penalty, both on. Nobody had, as far as I could find, separated those two design decisions and measured each one's contribution independently at fixed compute and matched evaluation. Policy A (both off), B (observation only), and C (penalty only) had to be built, and B and C are configurations upstream does not have. The second contribution is methodological rather than scientific: a deterministic 500-episode evaluation harness with a 20-column per-episode schema, which is what turned "the training curve went up" into "here is the number, here is its seed-to-seed spread, and here is which of my metrics you should not trust."

Q4. Why should a hiring panel care about a sim-only ablation?
Because the transferable skill on display is not "trained a policy" — policies are a tutorial outcome and the underlying task is public. It is the experimental discipline: proving the fork was bit-identical before modifying it, verifying the ablation actually removed the signal rather than down-weighting it, running three seeds and reporting the spread instead of the best run, and catching eight distinct instrumentation bugs that would each have produced a confidently wrong conclusion. One of them was a checkpoint-selection bug in RL-Games where the file named "best" had frozen mid-run because the reward scale changed discontinuously; shipping it would have meant evaluating a policy roughly ten points of success worse than the one I actually had. That kind of failure forensics is the part of the job that does not change between simulation and hardware.

Q5. If the conclusion had come out the other way, would you have published it?
The project spec anticipated exactly that and named the shape of an acceptable negative result up front — "force observations reduced jamming under pose perturbations but produced no meaningful improvement when the policy was already given exact relative geometry." And one arm did come out badly: Policy C, the penalty-only condition, performed **worse** than the geometry-only baseline. That is written in the README's headline finding rather than buried, because the interesting content of this study is in that cell. It is also the cell I trust least statistically, and I say that in the same breath.

2. Technical design choices

Q6. Why PPO? SAC is far more sample-efficient for continuous control.
SAC is more sample-efficient **per environment step**, which is the right metric when steps are expensive — on hardware, or in a slow simulator. Here steps are cheap and massively parallel: 512 environments on one GPU, roughly 1,150 simulated steps per second even on an aarch64 DGX Spark. In that regime the binding constraint is wall-clock throughput and stability under a huge, on-policy batch, which is what PPO with a large horizon (128 steps × 512 envs = 65k transitions per epoch) is good at. There are two more concrete reasons. PPO with an adaptive KL-targeted learning rate degrades gracefully when the reward scale shifts mid-run, which this reward does. And the upstream FORGE configuration is an RL-Games

PPO config, so starting there let me prove the fork bit-identical to upstream before changing anything — if I had swapped the algorithm on day one, I would have had no reference point to validate against. The honest caveat: I did not run a PPO-vs-SAC comparison, so I can defend the choice as reasonable, not as empirically superior on this task.

Q7. Why an LSTM actor-critic? Insertion looks Markovian if you have the poses.
It is not Markovian from the **actor's** point of view, and that is deliberate. Three quantities the policy needs are hidden from it: the per-episode proportional gains (randomized ±41%), the action EMA smoothing factor (sampled per episode from 0.025–0.1), and the per-axis action dead zone (re-randomized on a 2-second interval). All three change how a commanded setpoint turns into motion, none appear in the 24-dimensional actor observation, and all appear in the 61-dimensional critic state. The only way for the policy to compensate is to infer them from the history of commanded actions and observed responses — implicit online system identification. Recurrence is what makes that possible. The force channel adds a second reason: it is EMA-smoothed with $\alpha = 0.25$ and has 1 N of injected Gaussian noise, so a single-step reading is nearly uninformative and a short history is not. The configuration is two LSTM layers of 1024 units with layer norm, placed **before** the [512, 128, 64] MLP with the raw input concatenated through, unrolled over sequences of 128 steps — that is upstream's, inherited rather than tuned.

Q8. Explain the asymmetric actor-critic. Isn't giving the critic privileged information cheating?
It would be cheating if the critic were deployed, and it is not. At inference the critic is discarded entirely; only the 24-dimensional actor runs. The asymmetry is a variance-reduction trick: the value function's job is to estimate expected return from a state, and that estimate is far less noisy when computed from the true joint configuration, the true peg and socket poses, the true gains, and the clean force reading than from a noisy partial view. A better baseline means lower-variance advantage estimates, which means the policy gradient is less noisy — without the policy ever seeing anything it could not see on a robot. Concretely, the actor gets a noisy fingertip pose relative to a **noisy** socket-frame estimate and a noisy force; the critic gets clean versions of both plus `joint_pos`, `held_pos`, `held_quat`, `fixed_pos`, `fixed_quat`, `task_prop_gains`, `ema_factor`, and the control thresholds. This is standard practice in sim-trained manipulation for exactly this reason, and the deployability test is simple: does anything in the **actor** observation require a simulator to produce? Position relative to a socket estimate, orientation, end-effector velocities, wrist force, and previous actions are all available on a real cell with a pose estimate and an F/T sensor.

Q9. Walk me through the 24-dimensional actor observation and defend each choice.
In order: `fingertip_pos_rel_fixed` (3) — fingertip position minus the noisy estimate of the socket frame, so the policy sees an **error**, not a world coordinate, which prevents memorizing a fixed solution in table coordinates; `fingertip_quat` (4) — orientation, with two components zeroed because the task constrains roll and pitch, and with a randomly flipped global sign each episode so the policy cannot exploit quaternion double cover; `ee_linvel` and `ee_angvel` (3 + 3) — finite-differenced from the **noisy** fingertip pose, so their noise is correlated with the position channel exactly as it would be on a real differentiated estimate; `ft_force` (3) — EMA-smoothed wrist force rotated into the socket frame, plus 1 N Gaussian noise; `force_threshold` (1) — the per-episode randomized soft threshold in [5, 10] N above which the penalty applies; and `prev_actions` (7). Two of those deserve defense. The previous action matters because the controller has an EMA on commands and a dead zone: without knowing what it just asked for, the policy cannot reason about what the arm is currently doing. And feeding the force **threshold** as an observation is what makes the penalty learnable at all — the threshold is randomized per episode, so a policy that could not see it would be optimizing against a moving target it has no way to locate.

Q10. Why is there a 7th action dimension that is not a control output?

That is FORGE's success-prediction head: the policy outputs its own estimate of whether it has succeeded, rescaled from [−1, 1] to [0, 1], and the reward includes the absolute error against ground-truth success. Its purpose is to give a deployed policy a self-assessment signal — on a real cell you want the controller to tell you it is done rather than run to a timeout — and the environment logs precision, recall, and prediction delay for it at five confidence thresholds. Two honest notes. First, its penalty is **latched**: the scale stays at 0.0 and flips permanently to 1.0 the first time mean success crosses 25%. Second, that latch caused two real problems I had to diagnose, described later in this document, so I would not describe it as a free addition.

Q11. Why Isaac Lab / Isaac Sim rather than MuJoCo, Drake, or a custom PyBullet setup?
Three reasons, in order of weight. First, the FORGE peg-insertion task exists there with validated assets, a task-space impedance controller, and a wrist force sensor already wired to PhysX's link joint-reaction forces — reimplementing that faithfully elsewhere would have consumed the entire project budget and produced a less trustworthy baseline. Second, GPU-resident parallelism: 512 environments in one process with tensors that never leave the device, which is what makes 33M environment steps per seed and a twelve-checkpoint evaluation feasible on one machine. Third, reproducibility against a public reference — I could prove my fork bit-identical to the upstream implementation, which is a check that only exists if you stay in the same ecosystem. The counter-argument I would concede: MuJoCo's contact solver is arguably better-characterized for stiff contact, and if the research question were about contact **fidelity** rather than about what information a policy needs, that would be the stronger platform.

Q12. Why delta end-effector commands instead of joint torques?
Because the research question is about contact information, not about low-level control, and torque-level actions would force the policy to learn contact strategy and inverse dynamics simultaneously — with the result that a difference between Policy A and Policy B could be attributed to either. Keeping a fixed task-space impedance controller underneath holds the low-level layer constant across all four arms. Worth being precise about what the action actually is here, because it is not quite the spec's generic Δpose: the position action is a **target in the socket frame**, scaled by 5 cm bounds, and it is then clipped to a randomized per-episode threshold relative to the current fingertip pose. So the policy commands "where I want to be relative to the hole," and the clipping turns that into a bounded step. Roll and pitch are forced to zero, and yaw is remapped into a 270° span chosen to avoid the wrist joint limit.

Q13. You inherited the hyperparameters. Isn't that intellectually lazy?
It was a deliberate protocol choice, and the spec I was working to states it explicitly: reproduce the baseline before tuning, and do not tune ten hyperparameters before you have a reference point. There is also an experimental reason specific to an ablation — if I tuned per arm, every cross-arm difference would be confounded with tuning effort, which is a well-known way to manufacture whatever result you were hoping for. What I **did** change, and can defend on measurement, is the batch geometry: running 512 environments while leaving `minibatch_size` at the config's default of 512 produced 256 minibatches × 4 mini-epochs = 1024 gradient steps per epoch, eight times what the config was tuned for, and projected a 50-hour run. Overriding `minibatch_size` to 2048 restored the intended 32 minibatches per epoch and cut per-epoch cost from 181 s to 84 s. That is an engineering fix that preserves the optimization ratio, not a hyperparameter search.

3. Experimental design and ablation methodology

Q14. Describe the ablation precisely. What exactly differs between arms?

It is a full 2×2 factorial over two binary factors, implemented as two config flags on a single shared environment class rather than as four forked environments — which matters, because it means the four arms cannot silently drift apart. `use_force_obs` removes `ft_force` (3) and `force_threshold` (1) from both `obs_order` and `state_order` in `__post_init__`, taking the actor from 24 to 20 dimensions and the critic from 61 to 57. `use_force_penalty` zeroes the **raw** contact-penalty term — `relu(‖F‖ − threshold)` — rather than only its reward weight. A is both off, B is observation only, C is penalty only, D is both on and is bit-for-bit the upstream FORGE configuration. Everything else — assets, controller, randomization, PPO config, episode length, seeds within an arm, evaluation protocol — is held fixed.

Q15. Why zero the raw penalty term rather than just set its reward scale to zero?
Because setting the scale to zero would have left `logs_rew_contact_penalty` reporting the true unscaled contact force, and that curve is my **proof** that the ablation did what it claims. With the scale-only approach, an ablated run's chart is indistinguishable from a non-ablated one, and the only evidence the signal was removed is my assertion that I set a number to zero somewhere. With the raw term zeroed, Policy A's curve is flat at exactly 0.0 with zero variance across all 20 smoke-test iterations against a baseline fluctuating between 0 and 3.6 — a falsifiable artifact. The same logic drove the verification of the observation ablation: I confirmed it from the run's own TensorBoard event file showing the actor's input normalizer built at shape (20,) and critic at (57,), rather than trusting that a config flag had the effect I intended.

Q16. What does "bit-identical to upstream" actually prove — and what does it not?
It proves that the code path exercised by a 20-iteration, 64-environment run under a fixed seed produces the same final reward to six decimal places (22.362465) after being copied into an external project with a new task ID and a new package. That rules out the specific failure I was worried about: an accidental behavioural change during the copy — a dropped config field, a different default, an import resolving to a different version. It does not prove that upstream is correct, that longer runs would match, that untouched code paths behave identically, or that the physics is a good model of reality. And it is no longer true by design: adding the per-step `termination_reason` computation was an intentional deviation, and I record it as one rather than pretending the property still holds. It is a regression check with a defined scope, not a correctness proof.

Q17. Three seeds. Defend that as adequate.
I would not defend it as adequate; I would defend it as the honest limit of the compute I had, and be precise about what it does and does not support. Each seed is 13 hours of GPU time and the full batch was twelve runs, so three seeds per cell was roughly a week of continuous compute on a shared box. What that supports: reporting a range instead of a point, and noticing when an effect is larger than the range. Policy A's three seeds landed at 6.4 / 7.6 / 7.8% and Policy B's at 9.6 / 15.8 / 17.2% — non-overlapping, which is a meaningful observation even at n = 3. What it does not support: any significance claim, and specifically nothing about Policy C, whose seeds came in at 0.0 / 10.2 / 0.0%. That cell's **mean** of 3.4% is carried entirely by one seed and its median is zero. If I had one more week of compute, I would spend all of it on more seeds for C rather than on new conditions.

Q18. What is the actual statistical picture? Give me error bars.
Two variance sources, and they differ by an order of magnitude. Within a seed, 500 episodes at $p \approx 0.07$ gives a binomial standard error of about 1.2 percentage points — episode count is not the bottleneck. Between seeds, Policy B spans 7.6 points and Policy C spans 10.2. So seed variance dominates completely, and the correct unit of analysis is the seed, giving n = 3 per cell. With n = 3 and that spread, a formal test would be underpowered for everything except possibly the A-vs-B contrast. My position is that the observation effect is supported by non-overlapping per-seed ranges **and** by consistency across two

independent contrasts, and that the penalty effect is suggestive but not established. I would rather say that than quote a p-value from three points.

Q19. You have a 2×2. Did you actually analyse it as a factorial — main effects and interaction?
Yes, and it is the most interesting thing in the data — including a place where I would push back on my own writeup. Reading the four cell means as a factorial: the observation main effect is (B−A + D−C)/2 = (+6.9 + 6.5)/2 = **+6.7 points**, and the two estimates agree closely. The penalty main effect is (C−A + D−B)/2 = (−3.9 + −4.3)/2 = **−4.1 points**, and those agree too. The interaction on success rate is therefore only about −0.4 points, which is far below seed noise — meaning that **on success rate, the two effects are close to perfectly additive**. My own README says the effects are "not additive," and on success rate that phrasing overstates what the numbers show. Where non-additivity genuinely appears is contact force: the penalty removes 23.0 N of p95 force without the observation (A → C) but only 15.9 N with it (B → D), an interaction of about +7 N. A defensible interpretation is that the observation lets the policy **spend** force productively, so the penalty has less gratuitous force left to remove. All of this is arithmetic on three-seed means, and I would present it as a hypothesis the next batch should test, not a finding.

Q20. What confounds could invalidate this ablation? Argue against yourself.
Five, roughly in order of how much they worry me. **(1) Input dimensionality is confounded with information.** Removing the force channels changes the network's input width from 24 to 20, so Policy A and C differ from B and D in architecture, not only in information content. The clean fix is to keep the channel and feed zeros — which I did not do, because a constant channel gives the running-mean-std normalizer zero variance and needs special handling. Either choice has a defect; mine is the one I can point at. **(2) Equal compute is not equal convergence.** All arms got 500 epochs. If the force-aware arms converge more slowly, I have measured performance at a budget, not at asymptote. None of the training curves had clearly plateaued. **(3) Training seeds are not matched across arms.** Policy A used seeds 42/43/44; B, C, and D used 0/1/2. Since seeds are arbitrary draws from the same generator this does not bias anything, but it does mean no paired analysis is possible. **(4) Reward is not comparable across arms** — the penalty term changes the reward scale, so any cross-arm comparison must key off success rate and force, never `rewards/iter`. This is doubly true because of the success-prediction latch. **(5) The nominal suite is the training distribution**, deterministically seeded. It is a reproducibility control, not a held-out set.

Q21. What exactly does "deterministic evaluation" mean, and is the protocol identical across arms?
The evaluation script loads the checkpoint, forces the RL-Games player into `is_deterministic=True` — the mean action instead of a sample from the policy distribution, which matters because the YAML's own `player.deterministic` default is `False` — then runs the suite's episode budget as one re-seeded batch per suite seed, 64 environments at a time, and writes one CSV row per **completed** episode with all twenty §11.4 columns. Identical across all twelve checkpoints: the suite, the three seeds, the episode count, the determinism flag, the environment count. The only thing that differs is the task ID, which is what encodes the ablation. One subtlety worth volunteering: the environment's own randomization (per-episode gains, dead zone, EMA factor, contact threshold, mass) is still active during evaluation — "deterministic" refers to the policy and to the seeding, not to a frozen world.

Q22. How do you know your evaluation harness itself is correct?
Because I found four bugs in it before I trusted a single number from it, and each would have corrupted the results in a different direction. First, I had defined an episode as "done" when it succeeded **or** timed out — but resets only fire on timeout, so an environment that succeeded at step 50 kept reporting done every step through 149, writing roughly 100 duplicate rows for one episode. A 500-episode run would have been a handful of real episodes, massively over-weighted toward successes. Second, fixing that immediately

introduced the opposite bug: a peg that seated at step 50 and drifted by 149 was labelled a timeout, so I added sticky per-episode latches. Third, my first latch reused a buffer name the base class already owns for success-time logging, which would have corrupted upstream logic and its dtype. Fourth, a tensor rebind inside `torch.inference_mode()` turned an accumulator into an inference tensor and made the in-place reset fail. I also fixed a p95 that was truncating rather than interpolating its index — at small n it was reporting the 90th percentile under a p95 label. Then I validated the whole thing on a throwaway checkpoint: exactly 3 episodes per seed, 9 rows, 20 columns, no empty cells, 9 distinct values per metric column.

Q23. Why 500 episodes per checkpoint, and why those three seeds?
500 was chosen from the spec's suite definition and it is comfortably past the point of diminishing returns: at $p \approx 0.1$ the binomial standard error is about 1.3 points at $n = 500$ versus 1.9 at $n = 250$, while seed-to-seed spread is 7–10 points. Adding episodes buys precision on a quantity that is not the limiting uncertainty. The three suite seeds — 1000, 1001, 1002 — are fixed in `configs/evaluation_suites.yaml` and are deliberately disjoint from every training seed, so no policy was trained on the initial conditions it is scored against. One documented gap: Isaac Lab does not expose a per-episode seed, so the CSV's `episode_seed` column records the **batch** seed. A whole batch reproduces exactly; one arbitrary episode inside a batch does not reproduce in isolation. That is written into the script's docstring rather than left for a reader to discover.

4. Results and their interpretation

Q24. Your best policy succeeds 14% of the time. Why is that not a failed project?
It is a partial failure against the project's own target and I will not spin it: the spec set 80% nominal success as a strong version-1 result, and nothing here is close. What I would separate is the **absolute** claim from the **comparative** claim. The comparative claim — that the force observation approximately doubles success at fixed compute, fixed architecture, fixed reward, matched evaluation, and consistent across two independent contrasts in the factorial — does not depend on the absolute level being high. The absolute level has a mundane and testable explanation: 500 epochs is about 33M environment steps per seed, and none of the training curves had plateaued; contact-rich insertion from scratch is usually reported at an order of magnitude more experience than that. The right response is more training, not a reinterpretation, and I would rather report an undertrained honest number than tune until a number looks respectable.

Q25. Policy D is upstream FORGE's own configuration and it underperforms Policy B. Doesn't that suggest something is broken in your setup?
It is the single most surprising result in the batch and I treated it as a possible bug before treating it as a finding. What makes me think it is real rather than broken: Policy D was the **strongest** arm at training time by a wide margin — every seed reached double-digit final success, two exceeded 24%, with a peak of 30.9% — which is not what a broken configuration looks like. It is also the arm that needed no new code at all, being the pre-existing default task already proven bit-identical to upstream. And its evaluated results are the most consistent across seeds of any arm (9.2–10.8%). The gap is therefore between training-time and deterministic evaluation, not between working and broken. My working hypothesis, stated as a hypothesis: the contact penalty makes the policy conservative, and under stochastic exploration occasional risk-taking pays off across many rollouts in a way the deterministic mean action cannot reproduce. Testing that needs a noise-injected or stochastic-player evaluation, which is not built. The disciplined version of my claim is: at this compute budget and this evaluation protocol, D lands between the baseline and B on success while achieving the lowest force of any force-aware arm.

Q26. Policy C has the lowest contact force of all four. Isn't that the safety win you were looking for?

No, and reading it that way is exactly the trap the metric sets. Policy C's p95 force is 14.2 N against the baseline's 37.2 N, but its success rate is 3.4% against the baseline's 7.3%. A policy that mostly does not attempt insertion will produce beautiful force numbers, because low force here reflects under-exploration of contact-rich states rather than skilful careful contact. This is why the spec is emphatic that success and force metrics must be reported together, and why I would never quote a force number without the success number beside it. The genuine safety result in the batch is Policy D: 18.8 N — half the baseline's peak force — while **also** being above the baseline on success, with two of three seeds never once exceeding the 50 N threshold. That is a force reduction earned during actual insertions, not by declining to insert.

Q27. Your training curves and your evaluation numbers disagree. Which one is wrong?
Neither — they measure different things, and the discipline is in not conflating them. The training-time success rate is measured under the stochastic exploration policy against the continuously randomized training distribution, and is reported by RL-Games as a running quantity. The evaluation number is deterministic inference against fixed suite seeds. Policy A's final training success was 11–18% and its evaluated success 7.3%; Policy D's training peaked near 31% and evaluated at 9.9%. What the batch established is a rule I would now apply to any future work on this task: **training-time success is not a reliable rank-ordering of policies here.** Policy B looked unremarkable in training and evaluated best; Policy D looked best in training and evaluated third. If I had selected the "winning" configuration from training curves — which is what a lot of published ablations effectively do — I would have picked the wrong one.

Q28. Why is median completion time exactly 149 steps for every policy and every episode?
Because episodes never end early. `_get_dones()` returns an all-false `terminated` tensor by default, so every environment runs to the shared timeout and all environments reset together. Success and force-limit events are computed every step and **latched** as labels, but they do not end the episode — a peg that seats at step 50 keeps being simulated and keeps accruing reward through step 149. So "median completion steps," one of the spec's own primary metrics, is structurally uninformative on this fork, and I report it as an artifact rather than as a result. The knock-on effect worth stating in the same breath: "force-limit rate" is an **incidence** rate — the fraction of episodes in which force ever crossed the threshold — not a termination rate. Two different claims, and conflating them would inflate an apparent safety property.

Q29. Give me one number in your results you would defend hardest, and one you would defend least.
Hardest: Policy A at 7.3%. It is three independent seeds landing in a 1.4-point band (6.4 / 7.6 / 7.8), 1500 deterministic episodes, from a configuration whose ablation was independently verified through both the observation-size check and the flat-zero penalty curve, evaluated by a harness whose four known bugs were found and fixed before it produced this number. Its termination counts even cross-check: 109 successes out of 1500 pooled episodes is exactly 7.27%. Least: Policy C at 3.4%. Two seeds produced literally zero successes and one produced 10.2%; the mean is an artifact of averaging a bimodal outcome, and the median is zero. I report the mean because the whole table is means, but I flag it every time, and any statement about the penalty's main effect inherits that fragility.

5. Limitations, failure modes, and honest gaps

Q30. What is the biggest limitation of this work?
That the robustness half of the research question is untested. The question I set out to answer was whether force information improves **safety and robustness**, and robustness has an operational definition in the spec: success under pose shift, friction shift, mass shift, and a combined out-of-distribution suite, with a generalization gap computed against nominal. None of those suites are implemented — they are

commented-out stubs and the script raises `NotImplementedError` for them. So what I have is the safety half (contact-force distributions, which do differ substantially across arms) and the in-distribution success comparison. I would state that as: I have answered "does force information help the policy learn this task," and I have not yet answered "does it help the policy survive distribution shift." Building those suites is the single highest-leverage remaining item and it is already scoped.

Q31. Your domain randomization — is it actually doing what the config says?

Partly, and I only know which parts because of an accident. While implementing the friction columns of the evaluation CSV, I read back the effective friction values and both came back at exactly 0.75, which contradicted the `EventCfg` entry specifying a (0.25, 1.25) static-friction range for the fixed asset across 128 buckets. The explanation turned out to be that `FactoryEnv.__init__` calls `set_friction()` on all three assets **after** startup randomization, overwriting it with a constant from the task config. So that randomization range is dead config, and effective friction is a constant 0.75. That matters twice: it means my friction-robustness story is weaker than the config implies, and it is a ready-made cautionary example — the randomization you believe you have is a hypothesis until you read the value back out of the simulator. What **is** live and verified: per-episode controller gains (±41%), position and rotation clipping thresholds (25% / 29%), the action EMA factor, the action dead zone on a 2-second interval, the peg mass (±5 g), the contact-penalty threshold, initial pose, and observation noise on fingertip pose and force.

Q32. How well would this transfer to a real Franka?

I would not claim a number, and I would resist being drawn into one. What is genuinely transfer-oriented in the design, inherited from FORGE rather than invented by me: the actor never sees privileged state, the observation is noised, the force signal is smoothed and noised rather than clean, controller gains and dead zone are randomized so the policy cannot rely on one plant model, and the action space sits on top of an impedance controller rather than commanding torques. What is missing and would matter a great deal: no latency or actuation-delay randomization, no sensor drift or bias modelling on the force channel, no bandwidth model — the simulated force comes from PhysX link joint-reaction forces, which is not the same object as a strain-gauge F/T sensor with its own dynamics, temperature drift, and mounting compliance. The grasp is also idealized: the peg is pre-grasped and stays grasped, so slip in the fingers, a failure mode that dominates real insertion, is absent. My honest position is that this is a **study in simulation of what information a policy needs**, and its findings are hypotheses about hardware, not predictions.

Q33. A reviewer challenges your reward design as arbitrary. What do you concede and what do you defend?

I would concede the weights. The contact-penalty scale is 0.2 for peg insert, the asset action penalty 0.001, the action-gradient penalty 0.1 — those are upstream's numbers, I did not tune them, and I have no ablation over them. A reward with a hand-set weight vector is a modelling choice smuggled in as a constant, and the honest answer is that my study varies whether a term is present, not how strong it should be. What I defend is the **shape**: the contact penalty is `relu(‖F‖ − τ)` with τ randomized per episode in [5, 10] N, so it penalizes only force above a soft threshold rather than penalizing contact per se. That distinction is load-bearing — an all-contact penalty is precisely how you get a policy that refuses to touch anything, which is arguably what Policy C degenerated toward even with the threshold in place. I also defend the fact that every reward component is logged separately, so "which term dominates" is a chart lookup rather than a debate.

Q34. What are the actual observed failure modes?

The dominant one, by an enormous margin, is timeout with the peg near-seated. Pooled across Policy A's 1500 episodes: 1364 timeouts, 109 successes, 27 force-limit incidences, zero other. And the near-misses are genuinely near: the curated near-miss clips show final insertion depths around 1.5e-3 m against

successes at around 1e-4 m — roughly fifteen times the residual gap of a success, which sounds like a lot until you note that both are sub-millimetre. The success boundary is razor-thin, not a case of the policy being visibly lost. What I have **not** done is the spec's full failure taxonomy — rim collision, jamming, oscillation, over-caution, controller exploitation are named in the plan but not instrumented, and jam detection in particular has no signal and no threshold anywhere in the code. That is why the §22 jam-rate column reads `TBD` rather than 0%: "not instrumented" and "measured zero" are different claims and I refuse to let a table blur them.

Q35. Why did you not implement early termination, which your own spec requires?
I did implement it, as an opt-in experimental flag with per-condition counters, and then chose not to use it for any reported result — and the reasoning behind that choice is the part I would actually want an interviewer to see, because I got it wrong twice first. My first explanation, after a crash, was that per-environment reset was architecturally impossible; that was wrong, and the traceback actually showed a single one-line shape bug — a full-slice assignment receiving a partial-batch tensor — plus evidence that success and force-limit conditions were firing spuriously at episode start. My second explanation blamed a function that execution never reached. Both are recorded as retractions in the log rather than quietly edited away. The explanation that survived measurement is different and more interesting: `randomize_initial_state()` is a scripted IK-and-grasp routine that drives the **shared** PhysX world — it toggles global gravity to zero and back, issues un-indexed joint and root-pose writes that teleport every environment, and steps the simulator over a thousand substeps. Resetting a subset therefore perturbs the environments still running. Corroborating evidence: every Isaac Lab task that does do per-environment termination builds its reset purely from indexed buffer writes and never steps the simulator inside reset. So the synchronized-timeout design is a defensible scope decision with a measured justification, and the cost — a structurally meaningless completion-time metric and an incidence-rate rather than termination-rate force statistic — is documented in every results table.

Q36. The 50 N force limit — where did that number come from?
It is a placeholder and it is labelled as one in the code, in a comment that explains both what it is and why it was not changed. It is 5× the upper bound of the soft penalty band, chosen because no hard limit is specified anywhere in the project design. I did briefly have it at 15 N, from a measurement on a 20-iteration near-random checkpoint, and I reverted it — a near-random policy's force profile is not a sensible standard to judge a trained policy against, and silently changing what Policy A was evaluated against mid-project would have been worse than an acknowledged placeholder. The consequence is that "force-limit rate" is a number against an arbitrary threshold, so it is usable for ranking arms against each other and not usable as an absolute safety claim. On a real cell this number would come from the part's crush strength or the arm's rated payload, not from a multiple of a reward parameter.

6. Alternatives considered and rejected

Q37. Why not build the task from scratch instead of forking FORGE?
Because a from-scratch environment would have made every result unfalsifiable. If I had written my own peg-insertion task and Policy A had underperformed, I would have had no way to distinguish "geometry-only is worse" from "my environment has a bug." Forking a public, validated implementation gave me a reference point I could check against numerically, which is what the bit-identical regression test is for. The cost is that the environment is not my contribution, and I say so first rather than letting someone discover it.

Q38. Why not use a curriculum, which your own plan calls for?

The plan does specify a five-stage curriculum, from near-centred easy alignment through to full dynamics randomization, advancing on a rolling success threshold. I did not implement it, and the reason is comparability: a curriculum introduces a control loop between the policy's performance and the task distribution, which means two arms that learn at different rates are also **trained on different distributions**. In an ablation whose whole point is to attribute a performance difference to one factor, that is a confound I chose not to buy. The cost is probably a substantial amount of the missing absolute performance — starting under full randomization is a hard way to learn insertion. If I were optimizing for peak success rather than for a clean comparison, the curriculum would be the first thing I added.

Q39. Why not compare against a classical baseline — spiral search, admittance control?
I should have, and it is the clearest gap in the study; my own gap analysis ranks it as the second-highest-leverage addition after the evaluation suites, precisely because it is cheap. Without it I cannot say anything about whether RL is the right tool here, only about what information an RL policy needs. Two clarifications I would make in the same answer, though. First, this is not RL **instead of** classical control: there is a task-space impedance controller underneath, with gains, thresholds, and an action dead zone, and the policy commands its setpoints. So the architecture is already a hybrid. Second, the classical baseline that would actually be informative is a compliant spiral search with a hand-tuned admittance law under the same randomization and the same evaluation harness — anything less is a strawman, and a strawman comparison would be worse than no comparison.

Q40. Why not learn from demonstrations, or use offline RL?
For this project, because the question was about observation and reward design, and demonstrations change the learning problem so thoroughly that the ablation would no longer be measuring the same thing — a behaviour-cloned policy inherits the demonstrator's contact strategy whether or not it can sense force. As a follow-up it is attractive and cheap in simulation: scripted-expert demonstrations, behaviour cloning, then PPO fine-tuning, with the same evaluation harness, answers "do demonstrations reduce the environment steps needed for contact-rich insertion." That is one of the extensions named in the project plan, and it does not need teleop hardware.

Q41. Why not vision? Privileged pose observations are unrealistic.
They are, and the plan names vision as extension A for exactly that reason. The argument for deferring it is sequencing: adding pixels changes the sample complexity by roughly an order of magnitude and adds a representation-learning failure mode on top of a contact failure mode, so a negative result would be uninterpretable. Get the state-based version measured first, then ask how much robustness is lost when alignment must be inferred visually — with the state-based number as the ceiling to measure the loss against. I would also note that the actor's pose observation is relative to a **noisy** socket-frame estimate, which is a crude stand-in for exactly the error a perception system would introduce.

7. What I would do differently, and what is next

Q42. If you restarted this project tomorrow, what would you change?
Four things, in order. **(1) Build the evaluation harness before the first real training run.** I built it partway through and it immediately changed how I read every training curve. Every hour spent training before there is a trustworthy way to score the output is an hour spent generating numbers you will later have to re-interpret. **(2) Implement the ablation by zeroing channels, not removing them**, so all four arms share an identical network shape and the only difference is information content — with explicit handling for the degenerate normalizer. **(3) Fix the run-directory layout on day one.** All runs of all policies wrote into one shared experiment name with epoch numbers restarting at 1, which allowed a same-day prefix match to

pull unrelated smoke tests into a metrics extraction and fabricate a data point that did not exist. I caught it before publishing and fixed it with a manually verified run safelist, but the real fix is to never let the layout make that possible. **(4) Budget for more seeds and fewer conditions.** Given the seed variance I measured, five seeds on two conditions would have been more informative than three seeds on four.

Q43. What is the single next thing you would run?
The out-of-distribution suites — pose shift, low and high friction, mass shift, and the combined suite — evaluated on the twelve checkpoints that already exist. It is the highest-leverage remaining item for three reasons: it needs no new training, it completes the half of the research question that is currently unanswered, and it is the one place where I would expect the force-aware arms' advantage to **widen**, since force information should matter most exactly when the geometry prior is wrong. It would also produce the generalization-gap number the plan asks for. One prerequisite I would fix first: the fixed-asset friction randomization is currently overwritten by a constant, so a friction suite would silently measure nothing until that is corrected.

Q44. And after that?
In order of value per unit effort: instrument jam detection so the failure taxonomy stops being aspirational (sustained high force, negligible insertion progress, continued contact — the definition is already written down, only the threshold is missing); train past 500 epochs on at least the A and B arms to check whether the gap persists at convergence or is a transient of unequal learning speed; add the classical compliant-search baseline; and only then extend the study — either to multiple peg geometries, or to the auxiliary contact-prediction idea, where a model predicts short-horizon force or jam probability and that prediction is fed back as an observation. Sim-to-real sits behind all of these and behind hardware access.

Q45. What did you learn that you did not expect?
Three things. First, that the reward curve can be actively misleading in ways that have nothing to do with the policy: the success-prediction penalty latches on permanently the first time mean success crosses 25%, which drops total reward by roughly 84 points per episode in a single epoch and looks exactly like a collapse. It fired at epoch 302 in one seed and 269 in another, which means raw reward is not even comparable between two seeds of the **same** policy. Second, that this latch silently broke checkpoint selection — RL-Games only overwrites the "best" checkpoint when mean reward beats the stored best, so once the scale dropped permanently, the file labelled best froze mid-run and could never update, holding a policy about ten points of success worse than the final one. Third, and most general: that training-time metrics did not rank my policies the same way a deterministic evaluation did. Each of those is a case where the obvious number was the wrong number.

8. Curveballs and gotchas

Q46. Why not just use impedance control? Humans do this without reinforcement learning.
There **is** impedance control here — the policy commands setpoints to a task-space impedance controller with randomized gains, so the honest framing is RL on top of impedance control, not instead of it. The thing a fixed impedance law does not give you is the search-and-correct behaviour when the initial alignment is wrong by more than the chamfer will absorb, and the thing a hand-tuned law does not give you is adaptation when the plant changes — here, gains varying by ±41% episode to episode and a dead zone that shifts mid-episode. The fair rebuttal to my own answer is that I have not measured a classical baseline, so I cannot quantify the gap, and I would say so rather than assert RL's superiority. The intellectually honest position is that this project measures what information a learned controller needs, and leaves open whether a learned controller is the right choice at all.

Q47. Isn't "force feedback helps" a foregone conclusion? What if you had found nothing?
Partly foregone for the observation, not at all for the penalty — and I would point at the penalty result, because it is the one that is genuinely counterintuitive. The naive expectation is that a force penalty makes a policy safer. What actually happened is that a penalty without a corresponding observation made the policy **worse than the baseline** on success and produced its low force numbers by avoiding insertion. That is a real, non-obvious, and practically relevant finding: it says that a safety term in a reward is only meaningful if the agent can perceive the quantity being penalized, which sounds obvious stated that way and is violated constantly in practice. And a null result would have been publishable within this project's framing — the spec explicitly names "force gave no improvement because the policy already had exact relative geometry" as a legitimate conclusion.

Q48. How do you know your policy is not exploiting a simulator artifact?
I do not know it with confidence, and the fair answer starts there. What I have is four pieces of partial evidence. The success criterion is geometric, not reward-based — lateral error within tolerance and insertion depth past a threshold — so a policy cannot score by inflating a shaping term. The domain randomization spans controller gains, action smoothing, dead zone, mass, initial pose, and observation noise, which makes a solution that depends on one exact plant configuration harder to sustain. I watched curated videos of individual successes frame by frame and the peg is visibly hovering, then aligned, then seated — not teleported or squeezed through geometry. And the fingertip observation is noised, so a sub-millimetre exploit would have to survive noise larger than itself. What would actually settle it, and is not done: an out-of-distribution suite, a physics-parameter sweep (contact stiffness, solver iterations, timestep) to check the result is not solver-specific, and ultimately hardware. I would also flag the specific artifact I am most suspicious of — the force signal is a PhysX link joint-reaction force, not a modelled sensor, and I have not validated its magnitude against anything physical.

Q49. Isn't force feedback just a proxy for contact-rich manipulation more broadly? What generalizes?
The specific number does not generalize; the structure of the finding might. What I would claim travels is the decomposition itself: for any modality you might add to a manipulation policy — force, tactile, audio, proprioceptive residuals — you can and should separate "give the agent the signal" from "put a cost on the signal in the reward," because those two interventions are routinely bundled and they behaved very differently here. The observation was the factor that let the policy learn; the cost traded success for lower force. Whether the **magnitudes** transfer to a different task is untested, and the honest scope of my result is one task, one robot, one simulator, one compute budget.

Q50. You reported a mean of 3.4% for a cell containing two zeros. Is that not misleading?
It would be misleading if it stood alone, and it never does — the range appears next to every mean in every table, the per-seed rows are published in `results/tables/policy_c_nominal.md`, and the accompanying text states that two of three seeds failed to learn the task at all and that the pooled success is carried almost entirely by one seed. If I were writing a paper rather than a project log I would report the per-seed distribution as the primary object and the mean as a secondary summary, because for a bimodal cell the mean describes no run that actually happened. What that cell is genuinely evidence of is **instability**: the penalty-only configuration sometimes fails to learn at all, and that instability is itself a finding about the configuration.

Q51. Your seeds are not matched across arms — A used 42/43/44, B/C/D used 0/1/2. Does that not break the comparison?
It does not bias it, but it does cost me an analysis I would have liked. Seeds are arbitrary draws from the same generator, so there is no reason to expect a systematic difference between the set {42, 43, 44} and {0, 1, 2}; the arms are still independent samples from the same distribution over training runs. What it costs

is paired analysis — I cannot difference arm-by-arm within a matched seed, which would have removed some of the seed variance that currently dominates my error bars. The provenance is also worth telling straight: the plan document said to match A's seeds, the runs were launched with 0/1/2, and I found the discrepancy by cross-checking a monitoring dashboard's claim of "seed 42" against the run's own `params/env.yaml`. That is the same class of bug as the checkpoint freeze — a label that had drifted from the artifact it described.

Q52. Give me the most embarrassing bug you found.

Two candidates and I would offer both. The one with the largest potential to mislead: a metrics extraction that matched run directories by same-day prefix and silently pulled in unrelated smoke tests, whose epoch keys overwrote real data and produced an "epoch 499, reward 173.9" data point that never existed. It was caught before publishing and fixed with an explicitly verified run safelist. The one that cost the most time: the evaluation script exiting with status 0, no traceback, and no output file. Nothing was crashing — Isaac Sim's Kit has fast-shutdown enabled by default, so closing the simulation app terminates the process immediately, and **every line after the simulation context block silently never runs**. My CSV write was after the block. That one is worth telling because the failure signature was "success," which is the most dangerous kind.

Q53. Suppose I tell you this whole result is just noise. Convince me otherwise.

I would grant the framing for two of the four cells and contest it for one contrast. The A-versus-B contrast: per-seed ranges are 6.4–7.8 against 9.6–17.2 — disjoint, with every B seed above every A seed. Each seed is 500 deterministic episodes with a binomial standard error near 1.2 points, so within-seed noise cannot explain a 7-point separation. And the same effect appears independently in the second contrast of the factorial — C to D is +6.5 points against B minus A's +6.9, two estimates from disjoint data agreeing to within half a point. That coincidence is what I would rest on. What I would **not** contest: with n = 3 per cell I cannot give you a calibrated p-value, the penalty main effect rests on a bimodal cell, and the interaction term is well inside the noise. So my claim is "the observation effect is supported by converging evidence," not "the effect is statistically significant."

Q54. If you had unlimited compute tomorrow, what would you run first — and what would you expect?

A 5-seed × 4-arm batch trained to a plateau rather than a fixed 500 epochs, scored on the full suite of nominal plus four out-of-distribution conditions, with a stochastic as well as a deterministic player. My predictions, stated in advance so they can be wrong: absolute success rises substantially for every arm, because nothing had plateaued; the observation advantage narrows in-distribution as all arms saturate but **widens** under pose shift, because that is where the geometry prior degrades and force becomes the informative channel; and Policy D overtakes Policy B once conservatism stops costing it exploration. If the training-versus-evaluation inversion survives longer training, that is a more interesting finding than any of the above and it becomes the paper.

9. Fact sheet — numbers to have memorized

Item	Value
Task	`Shaurya-ForcePegInsert-Direct-v0` (= Policy D) plus `-PolicyA/B/C-Direct-v0`; fork of `Isaac-Forge-PegInsert-Direct-v0`
Stack	Isaac Sim 6.0.1, Isaac Lab v3.0.0-beta2.patch1, Python 3.12, RL-Games PPO
Hardware	DGX Spark `bas-zeus`, aarch64 Grace-Blackwell GB10, unified memory (so `nvidia-smi` cannot report GPU memory — RSS and `free` are the only signals)
Actor obs	24 dims: `fingertip_pos_rel_fixed` 3, `fingertip_quat` 4, `ee_linvel` 3, `ee_angvel` 3, `ft_force` 3, `force_threshold` 1, `prev_actions` 7. Policy A/C: 20
Critic state	61 dims: actor's clean equivalents plus `joint_pos` 7, `held_pos`/`held_pos_rel_fixed`/`held_quat`, `fixed_pos`/`fixed_quat`, `task_prop_gains` 6, `ema_factor`, `pos_threshold` 3, `rot_threshold` 3. Policy A/C: 57
Action	7 dims: 3 position targets in socket frame (±5 cm bounds), roll/pitch forced to 0, yaw remapped over 270° to dodge the joint limit, plus 1 success-prediction output
Controller	Task-space impedance; default prop gains [565, 565, 565, 28, 28, 28], per-episode noise ±41%; action EMA factor sampled from [0.025, 0.1]; dead zone up to [5, 5, 5, 1, 1, 1] re-randomized every 2 s
Force sensing	PhysX link incoming joint force on a `force_sensor` body, EMA-smoothed with α = 0.25, rotated into the socket frame, plus 1 N Gaussian observation noise
Contact penalty	`relu(‖F‖ − τ), τ` U[5, 10] N per episode, scale 0.2 for peg insert
Hard force limit	50 N — an untuned placeholder, 5× the soft band, labelled as such in code
Network	Shared trunk: LSTM 2 × 1024, layer-norm, before the MLP, input concatenated through → MLP [512, 128, 64], ELU. Separate central-value network of the same shape
PPO	γ = 0.995, GAE λ = 0.95, clip 0.2, horizon 128, 4 mini-epochs, adaptive LR from 1e-4 with KL target 0.008, grad-norm 1.0, entropy coef 0, value bootstrap, input and value normalization
Training budget	512 envs, `minibatch_size` 2048 (overridden from 512), 500 epochs ≈ 33M env steps/seed, 13 h/seed; Policy A's 3 seeds took 30.5 h wall-clock under GPU contention

Item	Value
Episode	`episode_length_s` = 10.0 → 149 policy steps; every environment resets together on timeout
Evaluation	`nominal` suite, 500 episodes/checkpoint, seeds `[1000, 1001, 1002]`, deterministic player, 64 envs, 20-column per-episode CSV
Pooled outcomes	Policy A: 1364 timeout / 109 success / 27 force-limit / 0 other. Policy D: 1351 / 149 / 0 / 0
Factorial effects	Observation +6.7 pts success; penalty −4.1 pts; success interaction ≈ −0.4 pts; force interaction ≈ +7 N

Phrases to avoid, and their honest replacements

Do not say	Say instead
“My policy is robust to domain shift.”	“I have not run the OOD suites yet; the robustness half of the question is open.”
“Force awareness improves safety.”	“The observation roughly doubled success; the penalty cut peak force by about half but cost some success. Policy C shows low force can mean avoidance, not safety.”
“The effects are not additive.”	“On success rate they are nearly additive — the interaction is about −0.4 points. The non-additivity is in contact force, around +7 N.”
“Median completion time is 149 steps.”	“Every episode runs to the timeout by design, so that metric is structurally uninformative on this fork.”
“Jam rate is 0%.”	“Jam detection is not instrumented — that column is `TBD`, not zero.”
“Statistically significant.”	“Non-overlapping per-seed ranges and two agreeing independent contrasts, at n = 3 seeds.”
“The fork is verified correct.”	“The fork reproduced upstream’s reward bit-identically on one 20-iteration seeded run; that is a scoped regression check, and it was intentionally broken later.”
“Domain randomization covers friction.”	“Fixed-asset friction randomization is overwritten by a constant 0.75 in `FactoryEnv.__init__` — dead config, and I found it by reading the value back.”

The framing that works. Not “I trained a peg-insertion policy” — that is a tutorial outcome and the underlying task is public. Instead: “I ran a controlled 2×2 force ablation across four policies and three seeds each, and caught eight instrumentation bugs that would each have produced a wrong conclusion — including a reward-

scale discontinuity that made a healthy training run look like a collapse, and a checkpoint-selection bug that would have shipped a policy ten points of success worse than the one I had.” The first is a claim about a model. The second is a claim about method, backed by file-and-line evidence.

The three things to volunteer before you are asked. (1) Absolute success is 7–14%, well under this project’s own 80% target, and the policies are undertrained rather than converged. (2) The evaluation is nominal-only, so nothing here measures out-of-distribution robustness yet. (3) Episodes never terminate early, which makes completion time meaningless and turns force-limit rate into an incidence rate. Stating these first converts them from “gotchas an interviewer found” into “limitations the candidate already understood.”

Sources: `force_aware_peg_insertion_project.md` (spec), `docs/experiment_log.md` (dated session log), `results/tables/all_policies_nominal.md` and the four per-policy tables, `results/raw/*.csv` (12 × 500 episodes), `force_peg_rl/source/.../tasks/direct/force_peg/` (environment, config, PPO YAML), and `docs/2026-08-13_agility_manipulation_role_gap_analysis.md`. Status as of 2026-08-18.